

Manual: Roteador Inteligente de LLMs Gratuitas

Arquivo: scripts/task_router.py **Versão:** 1.0.0 **Data:** 2026-08-17 **Compatível com:** Orca/MiMoCode, Claude Code, Aider, Cursor, Windsurf, Continue

line(length: 100%)

O que é

O roteador inteligente detecta o tipo de tarefa pelo seu prompt e escolhe automaticamente o melhor provedor de LLM gratuito disponível. Se um provedor estourar a cota, ele faz fallback para o próximo.

line(length: 100%)

Pré-requisitos

1. **Python 3.8+** instalado
2. **Arquivo .env** com as chaves dos provedores (já existe no projeto)
3. **python-dotenv** (opcional, mas recomendado):

```
pip install python-dotenv
```

line(length: 100%)

Passo 1 — Verificar provedores ativos

cd D:

```
\Backup_C_trcnologia_2026-08-14\Desktop\01_Projetos_e_Desenvolvimento\proj_fabrica-de-livros
```

```
python scripts/task_router.py --status
```

Saída esperada:

```
=====
ROTEADOR INTELIGENTE – STATUS DOS PROVEDORES
=====
● ATIVO | groq | uso: 0 | modelo: llama-3.3-70b-versatile
● ATIVO | google | uso: 0 | modelo: gemini-1.5-pro
...
Total ativos: 13/13
=====
```

Se algum provedor aparecer como ● INATIVO, adicione a chave no .env.

line(length: 100%)

Passo 2 — Testar o roteamento

Teste rápido (shell)

```
python scripts/task_router.py "debug this Python function"
```

Saída:

```
export ORCA_PROVIDER=groq
export ORCA_MODEL=llama-3.3-70b-versatile
```

Teste em JSON

```
python scripts/task_router.py "escreva um poema" --json
```

Saída:

```
{
  "task": "creative",
  "provider": "openrouter",
  "model": "meta-llama/llama-3.3-70b-instruct:free",
  "fallback": false
}
```

Teste legível (PT-BR)

```
python scripts/task_router.py "analise estes dados" --info
```

Saída:

```
Task: analysis
Provider: google
Model: gemini-1.5-pro
```

```
line(length: 100%)
```

Passo 3 — Usar no shell (recomendado)

Git Bash / Linux / Mac

Adicione ao ~/.bashrc ou ~/.zshrc:

```
# Roteador inteligente de LLMs
alias ai='eval $(python D:/Backup_C_trcnologia_2026-08-14/
Desktop/01_Projetos_e_Desenvolvimento/proj_fabrica-de-livros/scripts/task_router.py)'
```

Depois recarregue:

```
source ~/.bashrc
```

Uso:

```
ai "debug this function"
# → export ORCA_PROVIDER=groq
# → export ORCA_MODEL=llama-3.3-70b-versatile
```

```
orca # inicia com groq configurado
```

PowerShell

Adicione ao \$PROFILE:

```
function ai {
    $result = python D:
\Backup_C_trcnologia_2026-08-14\Desktop\01_Projetos_e_Desenvolvimento\proj_fabrica-
de-livros\scripts\task_router.py $args
    $result | ForEach-Object {
        if ($_ -match 'export (\w+)=(.+)') {
            [Environment]::SetEnvironmentVariable($matches[1], $matches[2],
"Process")
        }
    }
    Write-Host "Provider: $env:ORCA_PROVIDER | Model: $env:ORCA_MODEL"
}
```

Uso:

```
ai "escreva um poema"
# Provider: openrouter | Model: meta-llama/llama-3.3-70b-instruct:free
```

orca

line(length: 100%)

Passo 4 — Integrar com Orca/MiMoCode

Método A: Pré-sessão (recomendado)

1. Abra o terminal

2. Rote o comando:

```
eval $(python scripts/task_router.py "sua tarefa aqui")
```

3. Inicie o Orca:

```
orca
```

4. O Orca usará o provedor/modelo definidos

Método B: Wrapper

Crie um script start-ai.sh:

```
#!/bin/bash
cd D:/Backup_C_trcnologia_2026-08-14/Desktop/01_Projetos_e_Desenvolvimento/
proj_fabrica-de-livros
eval $(python scripts/task_router.py "$*")
echo "Provider: $ORCA_PROVIDER | Model: $ORCA_MODEL"
orca
```

Uso:

```
./start-ai.sh "debug this Python function"
```

Método C: Usar o router como referência

Se não quiser automação, rode o router e use a informação manualmente:

```
python scripts/task_router.py "minha tarefa" --info
# → Provider: groq | Model: llama-3.3-70b-versatile
# → Then select groq in Orca UI → Settings → Providers
```

line(length: 100%)

Passo 5 — Gerenciar quotas

Ver quanto já usou

```
python scripts/task_router.py --status
```

Mostra uso: N para cada provedor.

Resetar quotas

Se um provedor bloqueou por quota mas já liberou:

```
python scripts/task_router.py --reset
```

Onde fica o estado

```
~/task_router/quota.json    ← estado de quotas
~/task_router/usage.log     ← log de uso (append)
```

line(length: 100%)

Tabela de Roteamento

Tarefa	Provedores (ordem)	Modelo
coding	groq → cerebras → google → openrouter → nvidia	llama-3.3-70b-versatile
reasoning	google → groq → nvidia → openrouter → cerebras	gemini-1.5-pro
creative	openrouter → google → groq → siliconflow → nvidia	llama-3.3-70b-instruct:free
analysis	google → groq → cerebras → openrouter → nvidia	gemini-1.5-pro
chat	groq → google → openrouter → huggingface → siliconflow	llama-3.3-70b-versatile
embedding	huggingface → siliconflow → cohere → nvidia → fireworks	Phi-3.5-mini-instruct
vision	google → openrouter → nvidia → siliconflow → fireworks	gemini-1.5-pro

line(length: 100%)

Comandos de Referência

Comando	Descrição
python scripts/task_router.py "prompt"	Roteia e imprime export shell
python scripts/task_router.py "prompt" --json	Saída em JSON
python scripts/task_router.py "prompt" --info	Saída legível PT-BR
python scripts/task_router.py --status	Mostra provedores ativos e quotas
python scripts/task_router.py --reset	Reseta todas as quotas
python scripts/task_router.py --help	Mostra ajuda

line(length: 100%)

Solução de Problemas

“Provider: openrouter | fallback: true”

Significa que nenhum provedor da lista preferida tem chave. Verifique o .env.

Provedor bloqueado mas cota já liberou

python scripts/task_router.py --reset

python-dotenv não carrega o .env

Instale:

pip install python-dotenv

Ou exporte manualmente:

```
export $(cat .env | xargs)
```

Modelo não é suportado pelo Orca

O router define ORCA_MODEL, mas o Orca só aceita modelos da sua lista interna. Nesse caso, use --info e selecione manualmente no Orca UI → Settings → Providers.

line(length: 100%)

Arquivos Relacionados

Arquivo	Descrição
scripts/task_router.py	Script principal
scripts/detectar_llms_gratuitas.py	Detector de provedores (anterior)
.env	Chaves dos provedores
~/.task_router/quota.json	Estado de quotas
melhorias/2026-08-17-roteador-inteligente-llm.md	Plano de implementação